

Điểm khác biệt lớn nhất giữa module Slave này và module Master trước đó là **module này không có các tín hiệu điều khiển trực tiếp (backdoor access) như READ_EN, WRITE_EN hay address_memory**. Toàn bộ việc đọc/ghi vào RAM nội của module này bắt buộc phải thực hiện thông qua giao thức AXI chuẩn.

Dưới đây là phân tích chi tiết chức năng các nhóm tín hiệu và cách module này hoạt động:

1. Phân tích chức năng các nhóm tín hiệu (AXI Channels)

Các tín hiệu hoàn toàn tuân theo 5 kênh chuẩn của giao thức AXI (có lược bớt một số tín hiệu phức tạp về Burst Size/Type):

- **Kênh Ghi Địa Chỉ (Write Address Channel - AW):**
 - s_AWVALID_i / s_AWREADY_o: Cặp tín hiệu bắt tay (handshake). Master báo có địa chỉ hợp lệ (VALID), Slave báo sẵn sàng nhận (READY).
 - s_AWADDR_i, s_AWID_i, s_AWLEN_i: Nhận địa chỉ bắt đầu ghi, ID giao dịch và độ dài Burst từ Master.
 - **Kênh Ghi Dữ Liệu (Write Data Channel - W):**
 - s_WVALID_i / s_WREADY_o: Cặp bắt tay để truyền dữ liệu.
 - s_WDATA_i: Dữ liệu Master gửi đến để ghi vào RAM.
 - s_WLAST_i: Master báo đây là cờ dữ liệu cuối cùng trong 1 Burst.
 - **Kênh Phản Hồi Ghi (Write Response Channel - B):**
 - s_BVALID_o / s_BREADY_i: Slave phản hồi trạng thái ghi thành công (BRESP_o = 2'b00 tức là OKAY) cho Master sau khi đã nhận đủ dữ liệu Burst.
 - s_BID_o: Trả lại đúng ID của giao dịch ghi để Master quản lý.
 - **Kênh Đọc Địa Chỉ (Read Address Channel - AR):**
 - s_ARVALID_i / s_ARREADY_o: Cặp bắt tay cho việc gửi địa chỉ đọc.
 - s_ARADDR_i, s_ARID_i, s_ARLEN_i: Nhận địa chỉ cần đọc, ID và độ dài Burst từ Master.
 - **Kênh Đọc Dữ Liệu (Read Data Channel - R):**
 - s_RVALID_o / s_RREADY_i: Cặp bắt tay đẩy dữ liệu từ Slave về Master.
 - s_RDATA_o: Dữ liệu lấy từ RAM nội để trả về.
 - s_RLAST_o: Slave báo đây là dữ liệu cuối cùng trong Burst (burst_cnt == s_ARLEN_i).
-

2. Cách sử dụng (Hoạt động của Máy Trạng Thái FSM)

Để giao tiếp với module này trong testbench, bạn cần đóng vai trò là một AXI Master (hoặc kết nối nó trực tiếp với module `axi_master_if` ở câu trước).

Kịch bản 1: Ghi dữ liệu vào Slave (Write Transaction)

- Gửi Địa Chỉ:** Master kéo `s_AWVALID_i = 1` và cấp `s_AWADDR_i`. Máy trạng thái nhảy từ IDLE sang AW, Slave nhận địa chỉ và kéo `s_AWREADY_o = 1`.
- Gửi Dữ Liệu:** FSM chuyển sang WDATA. Master bắt đầu đẩy dữ liệu qua `s_WDATA_i` kèm `s_WVALID_i = 1`. Mỗi nhịp bắt tay, Slave sẽ lưu `s_WDATA_i` vào `mem[addr]`, tự động tăng con trỏ `addr` và bộ đếm `burst_cnt`. Ở data cuối, Master phải phát `s_WLAST_i = 1`.
- Nhận Phản Hồi:** FSM chuyển sang BRESP. Slave phát `s_BVALID_o = 1` báo hiệu ghi thành công. Master phản hồi `s_BREADY_i = 1` để FSM về lại IDLE.

Kịch bản 2: Đọc dữ liệu từ Slave (Read Transaction)

- Gửi Địa Chỉ:** Master kéo `s_ARVALID_i = 1` kèm `s_ARADDR_i`. FSM sang trạng thái AR, Slave ghi nhận địa chỉ và lưu `s_ARID_i`.
- Đọc Dữ Liệu:** FSM sang trạng thái RDATA. Slave liên tục kéo `s_RVALID_o = 1` và đẩy `mem[addr]` ra chân `s_RDATA_o`. Khi bộ đếm `burst_cnt` bằng với độ dài `s_ARLEN_i`, tín hiệu `s_RLAST_o` sẽ được kéo lên 1. Khi Master báo đã nhận xong bằng `s_RREADY_i = 1`, FSM về IDLE.

⚠ Lưu ý Cực Kỳ Quan Trọng (Phát hiện lỗi Logic)

Khi phân tích code, có một sự bất đồng bộ trong cách module này tính toán địa chỉ giữa kênh Đọc và kênh Ghi mà bạn cần cực kỳ lưu ý khi viết testbench:

- Khi Ghi (Trạng thái AW):** Code sử dụng `addr <= s_AWADDR_i[6:2];`. Điều này có nghĩa là module tự động dịch phải địa chỉ 2 bit (tương đương chia 4) để trỏ vào RAM theo đơn vị **Word** (32-bit).
- Khi Đọc (Trạng thái AR):** Code lại sử dụng `addr <= s_ARADDR_i;`. Module lấy trực tiếp địa chỉ mà không dịch bit.

Hậu quả: Nếu Master gửi địa chỉ `0x0000_0004` để ghi, nó sẽ được lưu vào vị trí `mem[1]`. Nhưng nếu Master gửi địa chỉ `0x0000_0004` để đọc, module lại đi đọc vị trí `mem[4]`. Dữ liệu ghi vào và đọc ra sẽ không khớp nhau nếu dùng định dạng địa chỉ Byte (Byte-addressable). Bạn nên sửa đổi `addr <= s_ARADDR_i[6:2];` ở nhánh AR để module hoạt động chính xác.